

ToggleMaster ❖❖❖ Guia de Execu❖❖❖o na AWS (Fase 4)

- [ToggleMaster — Guia de Execução na AWS \(Fase 4\)](#)
 - [Índice](#)
 - [1. Visão geral](#)
 - [2. Pré-requisitos](#)
 - [3. Etapa 1 — Credenciais AWS](#)
 - [4. Etapa 2 — Repositório Git](#)
 - [5. Etapa 3 — Bucket do Terraform](#)
 - [6. Etapa 4 — Provisionar a infra](#)
 - [7. Etapa 5 — Conectar kubectl](#)
 - [8. Etapa 6 — Secrets de observabilidade](#)
 - [9. Etapa 7 — Imagens Docker](#)
 - [10. Etapa 8 — ArgoCD](#)
 - [11. Etapa 9 — PagerDuty e Discord](#)
 - [12. Etapa 10 — Validar](#)
 - [13. Etapa 11 — Self-healing](#)
 - [14. Solução de problemas](#)
 - [15. Encerrar tudo](#)
 - [Resumo dos endereços e senhas](#)

ToggleMaster — Guia de Execução na AWS (Fase 4)

Guia passo-a-passo, do zero ao sistema funcionando, para iniciantes.

Este documento ensina como colocar o ToggleMaster completo rodando em um cluster **AWS EKS** usando uma sessão do **AWS Academy**, incluindo toda a stack de observabilidade da Fase 4 (Prometheus, Grafana, Loki, OpenTelemetry, Datadog, PagerDuty, Discord e self-healing).

Cada passo explica **o que** o comando faz e **por quê** ele é necessário, para que você entenda o processo e consiga resolver problemas se algo der errado.

Índice

1. [Visão geral do que vamos fazer](#)
2. [Pré-requisitos: ferramentas e contas](#)
3. [Etapa 1 — Preparar as credenciais da AWS Academy](#)
4. [Etapa 2 — Preparar o repositório Git](#)
5. [Etapa 3 — Criar o bucket do Terraform \(uma vez só\)](#)

6. [Etapa 4 — Provisionar a infraestrutura](#)
7. [Etapa 5 — Conectar o kubectl ao cluster](#)
8. [Etapa 6 — Criar os secrets de observabilidade](#)
9. [Etapa 7 — Construir e enviar as imagens Docker](#)
10. [Etapa 8 — Acompanhar o ArgoCD sincronizar tudo](#)
11. [Etapa 9 — Configurar o PagerDuty e o Discord](#)
12. [Etapa 10 — Validar que tudo funciona](#)
13. [Etapa 11 — Demonstrar o self-healing](#)
14. [Solução de problemas](#)
15. [Encerrar tudo \(importante!\)](#)

1. Visão geral

Vamos seguir este caminho:

Você escreve credenciais → Terraform cria a infra na AWS → EKS sobe → ArgoCD instala a aplicação + observabilidade → Você cria as contas (Datadog, PagerDuty, Discord) → Sistema funcionando → Demonstração do self-healing → Destruir tudo

O **Terraform** cria a infraestrutura (rede, cluster, bancos). O **ArgoCD** (que o Terraform instala) cuida de implantar os 5 microsserviços e toda a stack de monitoramento, lendo os manifestos do seu repositório Git (isso é o **GitOps**).

🕒 **Tempo total estimado:** 40 a 60 minutos, sendo ~20 min só esperando o EKS subir.

2. Pré-requisitos

2.1 Ferramentas que devem estar instaladas no seu computador

Ferramenta	Para que serve	Como verificar
AWS CLI (v2)	Falar com a AWS pelo terminal	<code>aws --version</code>
Terraform (≥ 1.6)	Criar a infraestrutura	<code>terraform version</code>
kubectl (≥ 1.28)	Comandar o cluster Kubernetes	<code>kubectl version --client</code>
Docker	Construir as imagens dos serviços	<code>docker --version</code>
Git	Versionar o código que o ArgoCD lê	<code>git --version</code>
Helm (≥ 3.13)	(opcional) inspecionar charts	<code>helm version</code>

Se algum comando acima der erro, instale a ferramenta antes de continuar.

2.2 Contas gratuitas que você precisa criar

Crie estas três contas **antes** de começar (são todas gratuitas):

1. **Datadog** — escolha o plano *For Education* ou *trial*. Anote depois a **API Key** e o **Site** (ex.: `us5.datadoghq.com`).
2. **PagerDuty** — plano *Developer* (gratuito). Você vai criar um *Service* e anotar a **Integration Key**.
3. **Discord** — crie um servidor (ou use um existente) e prepare um canal onde os alertas vão chegar.

🔔 Não se preocupe em configurar essas contas agora. A [Etapa 6](#) e a [Etapa 9](#) explicam exatamente onde pegar cada chave.

3. Etapa 1 — Credenciais AWS

O AWS Academy te dá credenciais **temporárias** que expiram quando a sessão do laboratório encerra. Você precisa copiá-las para o terminal toda vez que iniciar uma nova sessão.

Passo a passo:

1. Entre no AWS Academy e clique em **Start Lab**. Espere a bolinha ficar verde.
2. Clique em **AWS Details** → **AWS CLI** → **Show**.
3. Copie o bloco de credenciais mostrado.
4. No seu terminal, cole exportando como variáveis de ambiente:

```
export AWS_ACCESS_KEY_ID="ASIA...EXEMPLO"
export AWS_SECRET_ACCESS_KEY="wJaIr...EXEMPLO"
export AWS_SESSION_TOKEN="IQoJb3...EXEMPLO"
export AWS_REGION="us-east-1"
```

O que isso faz: todas as ferramentas (AWS CLI, Terraform, kubectl) leem essas variáveis para se autenticar na sua conta AWS. Sem elas, nada funciona.

Verifique se funcionou:

```
aws sts get-caller-identity
```

Se aparecer um JSON com `Account` e `Arn`, está autenticado. Se der erro de credencial, repita os passos (as credenciais podem ter expirado).

⚠ **Atenção:** essas credenciais expiram em ~3-4 horas. Se em algum momento os comandos começarem a falhar com “expired token”, volte aqui e copie credenciais novas do AWS Academy.

4. Etapa 2 — Repositório Git

O ArgoCD funciona lendo os manifestos Kubernetes de um repositório Git. Por isso o código precisa estar num repositório que o ArgoCD consiga acessar (público é o mais simples).

Passo a passo:

1. Crie um repositório no GitHub (ex.: `togglemaster-fase4`). Pode ser público.
2. Descompacte o projeto e suba para o repositório:

```
# Dentro da pasta do projeto descompactado
git init
git add .
git commit -m "ToggleMaster Fase 4 - entrega inicial"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/togglemaster-fase4.git
git push -u origin main
```

3. **Importante:** o código tem alguns lugares com a URL de exemplo `https://github.com/SEU-USUARIO/togglemaster-fase4.git` . Troque pela URL real do seu repositório nestes arquivos:

```
# Substitui a URL de exemplo pela sua, em todos os arquivos de uma vez
grep -rL "SEU-USUARIO/togglemaster-fase4" . | while read arquivo; do
    sed -i "s|https://github.com/SEU-USUARIO/togglemaster-fase4.git|https://github.com/SEU-USUARIO-REAL/SEU-REPO.git|g" "$arquivo"
done

git add . && git commit -m "ajusta URL do repositório" && git push
```

O que isso faz: o ArgoCD vai clonar esse repositório para descobrir o que instalar no cluster. Se a URL estiver errada, o ArgoCD não acha os manifestos e nada é implantado.

5. Etapa 3 — Bucket do Terraform

O Terraform guarda o “estado” da infraestrutura (o que ele já criou) em um arquivo. Para esse arquivo ficar seguro e compartilhável, guardamos num bucket S3. **Isso é feito uma vez só.**

Passo a passo:

```
# Escolha um nome único para o bucket (troque MEUID por algo seu, ex: seu RM)
export TF_BUCKET="togglemaster-tfstate-MEUID"

# Cria o bucket
aws s3api create-bucket --bucket "$TF_BUCKET" --region us-east-1

# Liga o versionamento (protege o estado contra perda)
aws s3api put-bucket-versioning --bucket "$TF_BUCKET" \
    --versioning-configuration Status=Enabled
```

O que isso faz: cria o “cofre” onde o Terraform anota tudo que criou, para na próxima vez saber o que já existe e o que falta.

Agora avise o Terraform sobre esse bucket. Abra o arquivo `terraform/backend.tf` e preencha:

```
terraform {
  backend "s3" {
    bucket = "togglemaster-tfstate-MEUID" # o mesmo nome de cima
    key    = "fase4/terraform.tfstate"
    region = "us-east-1"
  }
}
```

6. Etapa 4 — Provisionar a infra

Agora o Terraform vai criar tudo na AWS: rede (VPC), cluster EKS, bancos PostgreSQL (RDS), Redis (ElastiCache), filas (SQS), tabela (DynamoDB), repositórios de imagem (ECR) e o ArgoCD.

Passo a passo:

```
# Entre na pasta do terraform
cd terraform/

# Copie o arquivo de exemplo de variáveis
cp terraform.tfvars.example terraform.tfvars
```

Abra o `terraform.tfvars` e ajuste a linha do repositório:

```
aws_region = "us-east-1"
project     = "togglemaster"
cluster_name = "togglemaster-eks-prod"

gitops_repo_url = "https://github.com/SEU-USUARIO-REAL/SEU-REPO.git" # ← sua URL
gitops_revision = "HEAD"

expose_argocd_lb = false
```

Agora execute os três comandos do Terraform:

```
# 1) Baixa os plugins necessários (providers AWS, Kubernetes, Helm)
terraform init

# 2) Mostra o que SERÁ criado, sem criar nada ainda (revisão)
terraform plan

# 3) Cria tudo de verdade (confirme digitando "yes" quando pedir)
terraform apply
```

O que cada comando faz: - `init`: prepara o Terraform, baixando os “drivers” para falar com AWS/Kubernetes. - `plan`: faz um ensaio e lista tudo que vai ser criado. Use para conferir antes de gastar recursos. - `apply`: executa de fato. Vai pedir confirmação — digite `yes`.

⌚ **Isto demora ~20 minutos.** O gargalo é o EKS (o cluster Kubernetes), que a AWS leva tempo para provisionar. Pode tomar um café.

Ao terminar, o Terraform mostra as saídas (outputs). Guarde-as — você vai usar várias:

```
cluster_name = "togglemaster-eks-prod"
ecr_repository_urls = { auth = "...", flag = "...", ... }
argocd_initial_admin_password_command = "kubectl -n argocd get secret ..."
nlb_dns_command = "kubectl -n ingress-nginx get svc ..."
```

7. Etapa 5 — Conectar kubectl

O `kubectl` precisa saber como falar com o cluster que acabou de nascer. Este comando configura isso:

```
aws eks update-kubeconfig --name togglemaster-eks-prod --region us-east-1
```

O que isso faz: baixa as credenciais do cluster e as salva no seu `~/.kube/config`, para o `kubectl` saber onde e como se conectar.

Verifique se funcionou:

```
kubectl get nodes
```

Deve listar 2 nós (nodes) com status `Ready`. Se aparecerem, o cluster está no ar e acessível.

8. Etapa 6 — Secrets de observabilidade

A stack de monitoramento precisa de duas chaves secretas que **nunca** devem ir para o Git: a do Datadog e a do PagerDuty.

✂ **Há dois caminhos. Escolha um:**

Caminho A — automático (recomendado): se você definir estas chaves como *GitHub Secrets* no repositório, o workflow `terraform-infra.yml` cria os secrets no cluster sozinho, logo após o ArgoCD subir. Você não roda nenhum `kubectl` à mão. Veja a seção **8.0** abaixo.

Caminho B — manual: se preferir (ou se não usar o GitHub Actions para a infra), crie os secrets você mesmo com os comandos das seções 8.1 a 8.4.

Os dois caminhos produzem exatamente os mesmos secrets no cluster. O caminho A só move o trabalho do seu terminal para o pipeline.

8.0 Caminho A — definir os GitHub Secrets (automático)

No GitHub, vá em **Settings → Secrets and variables → Actions → New repository secret** e crie:

Nome do secret	Valor	Onde pegar
<code>DD_API_KEY</code>	a API key do Datadog	Datadog → Organization Settings → API Keys
<code>DD_SITE</code>	o site do Datadog (ex.: <code>us5.datadoghq.com</code>)	aparece na URL do seu Datadog
<code>PAGERDUTY_INTEGRATION_KEY</code>	a integration key do PagerDuty	PagerDuty → seu Service → Integrations → Events API V2

Pronto. Na próxima vez que o `terraform-infra.yml` rodar (push em `terraform/**` ou execução manual), o job **Post-Apply Validation** vai: 1. Esperar o ArgoCD ficar pronto; 2. Criar o `datadog-secret` a partir de `DD_API_KEY` / `DD_SITE`; 3. Aplicar a config do Alertmanager com a `PAGERDUTY_INTEGRATION_KEY`; 4. Forçar o ArgoCD a reconciliar a stack de observability.

Se algum secret não estiver definido, o workflow **não falha** — apenas emite um aviso e segue (você pode criar aquele secret manualmente depois, pelo caminho B).

⚠ Mesmo com o caminho A, a integração **PagerDuty → Discord** continua sendo manual (Etapa 9), porque é configurada na interface do PagerDuty e não há como automatizá-la via

8.x Caminho B — criar os secrets manualmente

Se optou pelo caminho A, **pule para a Etapa 7**. As seções abaixo são só para o caminho manual.

8.1 Onde pegar a chave do Datadog

1. Entre no Datadog → ícone de engrenagem (**Organization Settings**) → **API Keys**.
2. Copie uma **API Key** existente ou crie uma nova.
3. Anote também o **Site** (aparece na URL do seu Datadog, ex.: `us5.datadoghq.com`).

8.2 Criar o secret do Datadog no cluster

```
# Primeiro garante que o namespace exista
kubectl create namespace observability --dry-run=client -o yaml | kubectl apply -f -

# Cria o secret com a chave (troque pelos seus valores reais)
kubectl -n observability create secret generic datadog-secret \
  --from-literal=api-key="SUA_API_KEY_DO_DATADOG" \
  --from-literal=DD_API_KEY="SUA_API_KEY_DO_DATADOG" \
  --from-literal=DD_SITE="us5.datadoghq.com"
```

O que isso faz: guarda a chave do Datadog dentro do cluster, de forma que o Datadog Agent e o OpenTelemetry Collector consigam ler sem que a chave apareça em nenhum arquivo do Git.

💡 Por que dois nomes (`api-key` e `DD_API_KEY`)? O Helm chart do Datadog procura por `api-key`; o Collector lê `DD_API_KEY`. É a mesma chave, exposta com dois nomes para os dois componentes.

8.3 Onde pegar a chave do PagerDuty

1. No PagerDuty, vá em **Services** → **+ New Service** (ou use um existente).
2. Em **Integrations**, adicione uma integração do tipo **Events API V2**.
3. Copie a **Integration Key** gerada (uma sequência de ~32 caracteres).

8.4 Aplicar a config do Alertmanager com a chave do PagerDuty

O arquivo de configuração do Alertmanager tem um espaço reservado (`PAGERDUTY_INTEGRATION_KEY`) que você substitui no momento de aplicar:

```
# Volte para a raiz do projeto (saia da pasta terraform)
cd ..

# Guarde a chave numa variável
export PD_KEY="suaIntegrationKeyDoPagerDuty"

# Substitui o placeholder pela chave real e aplica no cluster
sed "s/PAGERDUTY_INTEGRATION_KEY/$PD_KEY/g" \
  gitops/base/observability/05-alertmanager-config.yaml \
  | kubectl apply -f -
```

O que isso faz: entrega ao Alertmanager a chave para abrir incidentes no PagerDuty quando um alerta dispara. O `sed` troca o texto-reservado pela chave real **só na hora de aplicar** — o arquivo no Git continua sem a chave (seguro).

9. Etapa 7 — Imagens Docker

Os 5 microserviços e o webhook de self-healing precisam virar imagens Docker e ser enviados para o ECR (o repositório de imagens da AWS que o Terraform criou).

✓ **Antes de tudo, valide que os serviços Go compilam** (recomendado):

```
cd services/auth-service      && go build ./... && cd ../../
cd services/evaluation-service && go build ./... && cd ../../
```

Se faltar `go.sum`, rode `go mod tidy` dentro de cada pasta antes.

Passo a passo:

```
# 1) Faz login no ECR (troque 604720765096 pelo SEU Account ID, visto no output do terraform)
aws ecr get-login-password --region us-east-1 \
| docker login --username AWS --password-stdin \
  SEU_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com

# 2) Constrói e envia cada imagem
for svc in auth flag targeting evaluation analytics self-healing-webhook; do
# Descobre a pasta certa (microserviços têm sufixo -service)
if [ -d "services/${svc}-service" ]; then
  PASTA="services/${svc}-service"
else
  PASTA="services/${svc}"
fi

TAG="v1.0.0-fase4-${git rev-parse --short HEAD}"
ECR_URL="SEU_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/togglmaster-${svc}"

echo "==> Construindo ${svc}..."
docker build -t "${ECR_URL}:${TAG}" "$PASTA"
docker push "${ECR_URL}:${TAG}"

# Atualiza o manifesto com a nova tag da imagem
ALVO="gitops/base/${svc%-webhook}"
[ "${svc}" = "self-healing-webhook" ] && ALVO="gitops/base/self-healing"
sed -i "s|image: .*togglmaster-${svc}.*|image: ${ECR_URL}:${TAG}|" \
  "${ALVO}/deployment.yaml" 2>/dev/null || true
done

# 3) Salva as tags atualizadas no Git (o ArgoCD vai ler isso)
git add gitops/base
git commit -m "fase4: atualiza tags das imagens"
git push
```

O que isso faz: transforma o código em imagens executáveis, envia para o ECR, e atualiza os manifestos para apontarem para essas imagens. O `git push` no final é o que “avisa” o ArgoCD que há algo novo para implantar.

🔔 Em um projeto real, o **GitHub Actions** faz tudo isso automaticamente a cada `push`. Aqui fazemos manual para você entender o fluxo.

10. Etapa 8 — ArgoCD

O ArgoCD é quem realmente implanta tudo no cluster, lendo do seu Git. Vamos abrir o painel dele para acompanhar.

Passo a passo:

Primeiro, **espere o ArgoCD ficar pronto** (isto evita o erro “connection reset by peer” que acontece se você tentar conectar antes do pod subir):

```
kubectl -n argocd wait --for=condition=ready pod \
  -l app.kubernetes.io/name=argocd-server --timeout=180s
```

Quando aparecer **condition met**, abra o túnel:

```
# Abre um túnel para o painel do ArgoCD (deixe rodando num terminal separado)
kubectl -n argocd port-forward svc/argocd-server 8080:80
```

```
# Em outro terminal, pegue a senha inicial do admin
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d ; echo
```

Agora abra no navegador: **http://localhost:8080** - Usuário: **admin** - Senha: a que apareceu no comando acima

⚠ Use **http://**, não **https://**. Este projeto configura o ArgoCD em modo **--insecure** (sem TLS) para simplificar. Se você mapeou a porta **:80**, o endereço é **http://localhost:8080**. Digitar **https://** aí resulta em erro de conexão.

🔗 Se der “connection reset by peer” ou “lost connection to pod”: veja a seção de [solução de problemas](#) — quase sempre é o pod ainda inicializando ou você usou o protocolo errado (http vs https).

No painel você verá várias “Applications”:

Application	O que é
auth, flag, targeting, evaluation, analytics	Os 5 microserviços
togglemaster-ingress	O ponto de entrada HTTP
observability-stack	A stack de monitoramento (Prometheus, Grafana, Loki, OTel, Datadog)
self-healing-webhook	O robô que reinicia serviços com problema

Espere todas ficarem **Synced** (verde) e **Healthy**. A **observability-stack** é a mais demorada (3 a 5 minutos), pois instala vários componentes.

Se alguma ficar travada em “OutOfSync”, force a sincronização:

```
kubectl -n argocd patch application observability-stack \
  --type merge -p '{"operation":{"sync":{}}}'
```

11. Etapa 9 — PagerDuty e Discord

Agora vamos conectar o PagerDuty ao Discord, para que os incidentes apareçam no seu canal. Isso é feito **na interface do PagerDuty** (não por comando).

Passo a passo:

1. **No Discord**, crie o webhook do canal:
 - Configurações do canal → **Integrações** → **Webhooks** → **Novo Webhook**.
 - Dê um nome (ex.: “Alertas ToggleMaster”) e **copie a URL do webhook**.
2. **No PagerDuty**, entre no seu **Service** → aba **Integrations** → **Add an Integration** → **Extensions**.
3. Escolha **Generic Webhook V2**, e configure:
 - **Name:** `Discord Notify`
 - **URL:** cole a URL do webhook do Discord e **acrescente `/slack` no final**.
 - Exemplo: `https://discord.com/api/webhooks/123/abc/slack`
4. Salve.

O que isso faz: sempre que o PagerDuty abrir um incidente (porque o Alertmanager mandou), ele vai disparar uma mensagem para o seu canal do Discord.

💡 Por que `/slack` no final? O PagerDuty envia a notificação no formato do Slack, e o Discord entende esse formato quando a URL termina com `/slack`. Sem isso, a mensagem não chega.

12. Etapa 10 — Validar

Vamos confirmar que cada parte está funcionando.

12.1 Todos os pods estão de pé?

```
kubectl -n observability get pods
```

Você deve ver pods como `kps-...` (Prometheus/Grafana), `loki-0`, `otel-...`, `datadog-...`, `alertmanager-...` e `self-healing-webhook-...`, todos `Running`.

12.2 Ver o dashboard do Grafana

```
# Túnel para o Grafana (deixe rodando)
kubectl -n observability port-forward svc/kps-grafana 3000:80
```

Abra **`http://localhost:3000`** - Usuário: `admin` - Senha: `postech2026`

No menu **Dashboards**, abra **“ToggleMaster — Visão Geral (Fase 4)”**. Você verá as 5 seções: estado dos serviços, requisições por segundo, erros, recursos do cluster e logs em tempo real.

12.3 Ver os logs no Loki

Ainda no Grafana, vá em **Explore** (menu lateral) → escolha a fonte **Loki** → digite a consulta:

```
{k8s_namespace_name="auth-namespace"}
```

Devem aparecer os logs do auth-service.

12.4 Gerar tráfego e ver no Datadog (APM e Service Map)

```
# Descobre o endereço público do sistema (o NLB)
INGRESS=$(kubectl -n ingress-nginx get svc ingress-nginx-controller \
-o jsonpath='{.status.loadBalancer.ingress[0].hostname}')
echo "Endereço: http://$INGRESS"

# Faz algumas requisições de teste
for i in $(seq 1 20); do
  curl -s "http://$INGRESS/evaluation/evaluate?user_id=uri&flag_name=feature_a" > /dev/null
done
```

💡 A URL usa `/evaluation/...` porque o ingress encaminha o prefixo `/evaluation` para o evaluation-service (removendo o prefixo antes de entregar ao serviço).

Agora abra o **Datadog** → **APM** → **Service Map**. Em 1-2 minutos, você verá os 5 serviços conectados por setas, mostrando que o trace percorre `evaluation → flag → targeting`.

13. Etapa 11 — Self-healing

Esta é a “prova real” exigida pela Fase 4: causar um problema de propósito e ver o sistema se curar sozinho.

Passo a passo:

```
# Terminal 1 – fique observando os pods do evaluation-service
kubectl -n evaluation-namespace get pods -w
```

```
# Terminal 2 – observe o log do robô de self-healing
kubectl -n observability logs deploy/self-healing-webhook -f
```

```
# Terminal 3 – quebre o evaluation-service de propósito
# (aponta para um serviço que não existe, gerando erros 5xx)
kubectl -n evaluation-namespace patch deployment evaluation-service \
--type='json' \
-p='[{"op": "replace", "path": "/spec/template/spec/containers/0/env/2/value", "value": "http://flag-quebrado:9999"}]'
```

O que vai acontecer (em 2 a 5 minutos):

1. O `evaluation-service` começa a responder erros 5xx.
2. O alerta **HighHttpRequestErrorRate** dispara (fica “Firing”) no Prometheus.
3. O **Alertmanager** envia para o **PagerDuty**, que abre um incidente.
4. O PagerDuty notifica o **Discord** (você vê a mensagem no canal).
5. O PagerDuty/Alertmanager chama o **webhook de self-healing**.
6. O webhook executa o equivalente a `kubectl rollout restart` no `evaluation-service`.
7. No **Terminal 1**, você vê os pods antigos terminando e novos subindo.

8. No **Terminal 2**, aparece a linha de log que é a **prova**:

```
{"msg": "auto_heal_executed", "service": "evaluation-service", "action": "rollout-restart", "ok": true, ...}
```

📺 **Para o vídeo/relatório:** capture (a) o alerta Firing no Prometheus/Grafana, (b) o incidente no PagerDuty, (c) a notificação no Discord, (d) o log `auto_heal_executed` e (e) os pods reiniciando. São as evidências exigidas.

Para restaurar o serviço ao normal depois da demonstração:

```
kubectl -n evaluation-namespace patch deployment evaluation-service \
  --type='json' \
  -p='[{"op": "replace", "path": "/spec/template/spec/containers/0/env/2/value", "value": "http://flag-service.flag-namespace.svc.cluster.local:8002"}]'
```

14. Solução de problemas

Sintoma	Causa provável	O que fazer
<code>aws</code> falha com “expired token”	Sessão do AWS Academy expirou	Copie credenciais novas (Etapa 1)
<code>terraform apply</code> falha na metade	Sessão expirou durante o apply	Renove credenciais e rode <code>terraform apply</code> de novo (ele continua de onde parou)
<code>kubectl get nodes</code> mostra “Unauthorized”	kubeconfig desatualizado	Rode de novo <code>aws eks update-kubeconfig ...</code> (Etapa 5)
ArgoCD App “observability-stack” travada em OutOfSync	CRDs do Prometheus demoram	Aguarde 5 min ou force sync (Etapa 8)
Pod <code>datadog-cluster-agent</code> em CrashLoop	API Key do Datadog errada	Confira o secret: <code>kubectl -n observability get secret datadog-secret -o yaml</code>
Alertmanager: “no configuration loaded”	Esqueceu de substituir a chave do PagerDuty	Refaça o <code>sed ... kubectl apply</code> (Etapa 6.4)
Discord não recebe nada	URL do webhook sem <code>/slack</code> no final	Edite a Extension no PagerDuty (Etapa 9)
Dashboard do Grafana vazio	Sem tráfego ainda	Gere requisições (Etapa 10.4) e aguarde ~1 min
Self-healing não reage	RBAC ou label faltando	<code>kubectl -n observability logs deploy/self-healing-webhook</code> para ver o erro
ArgoCD: “connection reset by peer” / “lost connection to pod”	Pod ainda inicializando	Espere ficar pronto: <code>kubectl -n argocd wait --for=condition=ready pod -l app.kubernetes.io/name=argocd-server --timeout=180s</code> e tente

		de novo
ArgoCD: página não abre / erro de conexão no navegador	Usou <code>https://</code> com a porta <code>:80</code> (modo insecure é http)	Acesse <code>http://localhost:8080</code> (com <code>:80</code>), ou troque para <code>8080:443</code> e use <code>https://</code>
ArgoCD: port-forward cai repetidamente	Instabilidade no service	Encaminhe direto no pod: <code>kubectl -n argocd port-forward deploy/argocd-server 8080:8080</code> → <code>http://localhost:8080</code>

Comandos úteis de diagnóstico:

```
# Ver todos os pods de um namespace e seus status
kubectl -n observability get pods

# Ver por que um pod não sobe (eventos no final da saída)
kubectl -n observability describe pod NOME-DO-POD

# Ver os logs de um pod
kubectl -n observability logs NOME-DO-POD
```

15. Encerrar tudo

⚠ **MUITO IMPORTANTE:** o AWS Academy tem créditos limitados. Sempre destrua a infraestrutura quando terminar, ou os créditos acabam.

```
# Volte para a pasta do terraform
cd terraform/

# Destrói tudo que foi criado (confirme com "yes")
terraform destroy
```

O que isso faz: remove o cluster EKS, bancos, filas, rede — tudo. Demora ~15 minutos (a VPC e o EKS são lentos para remover).

💡 O bucket S3 do estado (Etapa 3) **não** é destruído pelo `terraform destroy` e pode ser reaproveitado na próxima vez. Se quiser remover também: `aws s3 rb s3://$TF_BUCKET --force`.

Resumo dos endereços e senhas

O quê	Como acessar	Credenciais
ArgoCD	<code>kubectl -n argocd port-forward svc/argocd-server 8080:80</code> → <code>http://localhost:8080</code>	admin / (comando da Etapa 8)
Grafana	<code>kubectl -n observability port-</code>	admin / postech2026

	<code>forward svc/kps-grafana 3000:80</code> → http://localhost:3000	
Prometheus	<code>kubectl -n observability port-forward svc/kps-kube-prometheus-stack-prometheus 9090:9090</code> → http://localhost:9090	—
Aplicação	<code>kubectl -n ingress-nginx get svc ingress-nginx-controller</code> (pegue o hostname)	—

Documento gerado para o Tech Challenge Fase 4 — PosTech / FIAP. Para detalhes de arquitetura, consulte [ARCHITECTURE.md](#) ; para a lista de mudanças, [CHANGES.md](#) .